

A. Real-World Impact

To study the prevalence of logic discrepancies in real-world mini-programs, we randomly collected 10,000 WeChat mini-programs using MiniCrawler [12], covering a wide range of categories such as business, utilities and entertainment. To analyze these mini-programs, we developed a heuristic-based static analysis tool designed to identify signatures of the root causes of these logic discrepancies. Take the Root Cause ③ (inconsistent Object (de)serialization) as an example. One of the related issues (Issue P5 in Table II) involves Channel ⑨ (XML $\xleftrightarrow{ev}$ EJS). The signature of this issue is that a EJS function is used as the event handler for a XML page element.

The analysis results show that 793/10000 (7.93%) mini-programs contain inconsistent-prone signatures, with 5.54% involving Root Cause ①, 5.30% involving Root Cause ②, 38.21% involving Root Cause ③, 51.03% involving Root Cause ⑤. Note that even if a mini-program contains inconsistency-prone signatures, observable inconsistent behaviors may not manifest due to the absence of explicit observation oracles or the trigger condition is not satisfied. However, such a mini-program still carries the risk of inconsistent functionalities, regardless of observability. Here, we present two case studies that demonstrate the real-world security implications of logic discrepancies in cross-layer communication. **Unauthorized Access.** We use a real-world family-tree mini-program (AppID: wxf02b444b2c134ee2) as a case study to demonstrate unauthorized access resulting from logic discrepancies in cross-layer communications. Figures 1(b) and 1(c) show the rendered Home page on Android and Windows, respectively. The bottom tab bar should not be visible when the user is not logged in, as on Android, but it is displayed on Windows. As a result, users can access the Circle page (Figure 1(d)) containing family-tree details, which is intended to be available only to logged-in users.

Further inspection reveals that this discrepancy is similar to Issue C10 in Table II with Root Cause ④ (confused messaging channel). As shown in Figure 1(a), the bottom tab bar is defined in both `app.js` and `tabbar.js`, and the corresponding data are stored in variables with the same name. This name collision confuses the messaging channel when `tarbar.list` is transmitted across layers, making it ambiguous whether it refers to the `list` variable defined in `tabbar.js` or the `list` field of the `tarbar` object defined in `app.js`. The former enforces an accessibility check based on login status (Line 1 in Figure 1(a)), whereas the latter does not. Android prioritizes the former, whereas Windows prioritizes the latter, which explains the discrepancy.

```

tabbar.js:
1. accessibility: { attached: function () { return loginSate } },
2. data: {
3.   color: "var(--text-color-2)",
4.   selectedColor: "var(--color-primary)",
5.   list: [
6.     {pagePath: "/pages/home/home", text: "我的家族"},
7.     {pagePath: "/pages/circle/circle", text: "家族广场"},
8.     {text: "发布"},
9.     {pagePath: "/pages/message/message", text: "消息"},
10.    {pagePath: "/pages/user/user", text: "个人"}
11.  ],
12. },
app.js:
13. "tabBar": {
14.   "custom": true, "color": "#6A6A6A",
15.   "selectedColor": "#409EFF",
16.   "borderStyle": "white", "backgroundColor": "#ffffff",
17.   "list": [
18.     {"pagePath": "pages/home/home", "text": "首页"},
19.     {"pagePath": "pages/circle/circle", "text": "家族圈"},
20.     {"pagePath": "pages/message/message", "text": "消息"},
21.     {"pagePath": "pages/user/user", "text": "我的"}
22.   ]
23. },

```

(a) related code snippet



(b) Home on Android



(c) Home on Windows



(d) Circle on Windows

Fig. 1: Unauthorized access resulting from logic discrepancies in cross-layer communications.